

The **SOLID principles** are a set of five design principles in object-oriented programming intended to make software designs more understandable, flexible, and maintainable. They are especially useful in **clean coding**, **refactoring**, and **system architecture**.

✅ SOLID Acronym:

Principle	Meaning
S – Single Responsibility	A class should have only one reason to change
O – Open/Closed	Software entities should be open for extension, but closed for modification
L – Liskov Substitution	Subtypes must be substitutable for their base types
I – Interface Segregation	Prefer many specific interfaces over one general-purpose interface
D – Dependency Inversion	Depend on abstractions , not concrete implementations

🔍 Detailed Explanation with Example (in C#)

1. Single Responsibility Principle (SRP)

Each class should have only one responsibility.

❌ Bad:	✅ Good:
<pre>class Invoice { public void CalculateTotal() { /*...*/ } public void PrintInvoice() { /*...*/ } // printing responsibility public void SaveToDatabase() { /*...*/ } // persistence responsibility }</pre>	<pre>class InvoiceCalculator { public void CalculateTotal() { /*...*/ } } class InvoicePrinter { public void PrintInvoice() { /*...*/ } } class InvoiceRepository { public void SaveToDatabase() { /*...*/ } }</pre>

2. Open/Closed Principle (OCP)

Entities should be open for extension but closed for modification.

✔ Use inheritance or interfaces to extend functionality.

```
abstract class Shape {  
    public abstract double Area();  
}  
  
class Circle : Shape {  
    public double Radius { get; set; }  
    public override double Area() => Math.PI * Radius * Radius;  
}  
  
class Rectangle : Shape {  
    public double Width { get; set; }  
    public double Height { get; set; }  
    public override double Area() => Width * Height;  
}
```

3. Liskov Substitution Principle (LSP)

Derived classes must be replaceable for their base class without breaking the app.

✔ Good: <pre>class Bird { public virtual void Fly() {} } class Sparrow : Bird { public override void Fly() { /* flies */ } }</pre>	✗ Violates LSP: <pre>class Ostrich : Bird { public override void Fly() { throw new Exception("Can't fly"); } }</pre>
--	---

4. Interface Segregation Principle (ISP)

No client should be forced to implement methods it doesn't use.

❌ Bad: <pre>interface IWorker { void Work(); void Eat(); // Not every worker needs this }</pre>	✅ Good: <pre>interface IWorkable { void Work(); } interface IEatable { void Eat(); }</pre>
--	--

5. Dependency Inversion Principle (DIP)

High-level modules should not depend on low-level modules. Both should depend on abstractions.

❌ Tight coupling: <pre>class LightBulb { public void TurnOn() { } } class Switch { private LightBulb bulb = new LightBulb(); public void Operate() { bulb.TurnOn(); } }</pre>	✅ Use abstraction: <pre>interface IDevice { void TurnOn(); } class LightBulb : IDevice { public void TurnOn() { } } class Switch { private IDevice device; public Switch(IDevice device) { this.device = device; } public void Operate() { device.TurnOn(); } }</pre>
---	--

Summary

- **S**: One responsibility per class
- **O**: Extend without modifying
- **L**: Replace base class with derived class safely
- **I**: No fat interfaces
- **D**: Depend on abstractions

Single Responsibility Principle (SRP)

Q1: If breaking down a class into smaller ones increases the number of files, is that always better?

A: Not necessarily. SRP is about **logical separation of responsibilities**, not file count. I break classes only when there is a clear and meaningful separation, like splitting business logic, persistence, and presentation. Over-splitting can lead to unnecessary complexity.

Q2: Can a class have more than one method if it follows SRP?

A: Yes. A class can have multiple methods if they all serve **one cohesive responsibility**. For example, a ReportGenerator class can have GeneratePdf() and GenerateCsv() if both relate to report generation.

Q3: How do you decide what counts as a “responsibility”?

A: I use the “reason to change” test — if a change request can affect a class for multiple unrelated reasons, it has more than one responsibility.

Q4: Have you ever refactored a God class?

A: Yes. In a .NET API, a CustomerService was handling validation, DB calls, and email notifications. I split it into CustomerValidator, CustomerRepository, and NotificationService. This reduced change impact and improved test coverage.

Open/Closed Principle (OCP)

Q5: How do you balance OCP with avoiding over-engineering?

A: I only introduce extension points when I see a **real likelihood** of change. For example, I won’t create a strategy pattern for something that’s unlikely to change in the foreseeable future.

Q6: How would you add new features without changing existing code?

A: By using abstractions. For example, in a payment processing system, I’d add a new IPaymentProcessor implementation instead of modifying the existing one.

Q7: In your payment gateway example, what happens if the interface changes?

A: Changing the interface affects all implementations, so I try to design the interface to be **stable** and keep changes additive (adding new interfaces rather than breaking existing ones).

Q8: How does OCP relate to the Strategy pattern?

A: The Strategy pattern supports OCP by allowing algorithms to be selected at runtime without modifying the client code.

Liskov Substitution Principle (LSP)

Q9: Can you give a real-world example where violating LSP caused issues?

A: In one project, a subclass `ReadOnlyFile` overrode `Write()` to throw `NotSupportedException`. This broke code that expected every `File` type to be writable, causing runtime errors.

Q10: Is LSP relevant if you only use interfaces?

A: Yes. Even with interfaces, your implementations must respect the **contract**. If an interface says a method works a certain way, all implementations should behave accordingly.

Q11: What's the "Ostrich problem" in LSP?

A: It's when a subclass inherits a method but throws `NotImplementedException` instead of providing a real implementation. This violates LSP because the subclass can't fully substitute the base class.

Q12: If a subclass throws `NotImplementedException` for a base method, is that an LSP violation?

A: Yes, unless it's clearly documented and expected by the interface contract. But generally, it means the hierarchy design is flawed.

Interface Segregation Principle (ISP)

Q13: What's the downside of having too many small interfaces?

A: Too many small interfaces can lead to **interface bloat** and make the system harder to navigate. The goal is balance — not one huge interface, but also not dozens of one-method interfaces.

Q14: Can splitting interfaces too much cause problems in a large team?

A: Yes, it can cause **coordination overhead** and make onboarding harder. That's why I group methods that are **logically cohesive** in one interface.

Q15: How is ISP related to microservices design?

A: In microservices, services should expose **small, focused APIs** — similar to ISP, which ensures clients only depend on what they need.

Q16: Have you ever merged two interfaces back into one?

A: Yes, when two interfaces kept evolving together and were always used in the same contexts, merging made sense to reduce redundancy.

Dependency Inversion Principle (DIP)

Q17: How is DIP different from dependency injection?

A: DIP is a **design principle** — high-level modules depend on abstractions, not concrete classes. Dependency Injection (DI) is a **technique** to implement DIP by supplying dependencies at runtime.

Q18: If you use DI everywhere, are you automatically following DIP?

A: Not necessarily. If you inject **concrete classes** instead of abstractions, you're still tightly coupled and violating DIP.

Q19: Can DIP cause performance issues?

A: In high-frequency operations, extra layers of abstraction can cause minor overhead. I mitigate this with profiling and only simplify layers where performance is critical.

Q20: In Angular, how does the provider system support DIP?

A: Angular's DI framework lets components depend on **interfaces (via tokens) or abstract classes** instead of concrete services, enabling loose coupling and easy swapping of implementations.

Principle Trick / Cross Question**Senior-Level Short Answer**

SRP	Does splitting into many classes always follow SRP?	No. SRP is about having one reason to change, not class count. Over-splitting adds complexity.
	Can a class have multiple methods under SRP?	Yes, if they all serve one cohesive purpose.
	How do you identify a “responsibility”?	Use the "reason to change" rule.
OCP	How to avoid over-engineering when following OCP?	Add extension points only when you anticipate real change.
	How to add features without modifying existing code?	Use abstractions and implement new classes (e.g., Strategy pattern).
	If interface changes, doesn't that break OCP?	Yes, so design stable interfaces and evolve via new ones.
LSP	Is LSP relevant if you only use interfaces?	Yes. Implementations must respect the interface contract.
	What's the Ostrich problem?	Subclass inherits a method but throws NotImplementedException — violates LSP.
	Is throwing NotImplementedException an LSP violation?	Yes, unless explicitly documented and expected.
ISP	Can too many small interfaces be a problem?	Yes. Leads to interface bloat — balance is key.
	How does ISP relate to microservices?	Both ensure clients depend only on what they need.
	Have you merged interfaces before?	Yes, when they always evolve and are used together.
DIP	Is DI the same as DIP?	No. DIP is a principle, DI is a technique to achieve it.
	Does using DI everywhere guarantee DIP?	No, injecting concrete classes still violates DIP.
	Can DIP impact performance?	Slightly in hot paths — profile and simplify only where needed.
	How does Angular support DIP?	Via its DI framework, letting components depend on tokens/abstractions instead of concretes.